Name:

COMP 4537 Midterm Sheet · Side 1

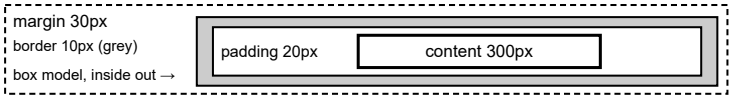
HTML/CSS/JS · scope & hoisting · event queue · JSON & Web Storage · architecture · REST. *[real: …]* = true behaviour where the slide differs.

HTML (L1)

- `<!DOCTYPE html>` = HTML5; `<html>` root, `<head>` meta info, `<body>` visible. HTML creates elements; **DOM** (tree) = how they attach; CSS = where/how they look; JS = interactivity.
- Single (empty) tags `<img> <br> <hr>`. Global attributes `style id class`: `id` unique per page, `class` need not be.
- **Block**: `div h1–h6 p form header footer section`. **Inline**: `span a img`. `<strong>/<em>` semantic importance; `<b>/<i>` none.
- Radio buttons with the same `name` = one group. **URL, 4 parts**: protocol, domain name, folder path, file name; no file name → `index.html`.

CSS (L1)

- **Inline** (style attr), **internal** (`<style>` in head), **external** `<link rel="stylesheet" href="my.css">`. Rule-set = selector + declaration block `{ name: value; }`.
- `p` · `#firstId` · `.firstClass` · `p.myClass` (only <p>) · `h1, h2` group. **Combinators**: `div p` (space) descendant, `>` child, `+` adjacent sibling, `~` general sibling. id > class > element.
- `display:none` frees the space; `visibility:hidden` keeps it (both stay in the DOM). **position**: static (default, document flow), relative, absolute, fixed (browser window), sticky.
- pt = 1/72 in; **em** = element's font-size; **%** = parent. RGB 0–255, alpha 0 transparent – 1 opaque.



JS values (L1)

- **6 primitives**: Boolean, null, undefined, Number, String, Symbol; + Object (arrays too: `typeof []` → "object"). `==` converts types; `===` does not (type, then value).

`[10] == 10` true	`'10' == 10` true	`[] == 0` true
`[10] === 10` false	`'10' === 10` false	`'' == false` true
`true == "a"` false	`'' === false` false	`[] === 0` false

- **Falsy (6)**: `undefined null 0 false NaN ''` (`"0" [] {}` are truthy). `undefined` = uninitialized; `null` = cleared on purpose; undeclared → ReferenceError "x is not defined".
- Identifier starts with letter, _, $ (`2a` SyntaxError). Outputs: `document.write()`, `console.log()`, `window.alert()` (bad UX), `innerHTML`.
- `typeof`: 42 "number", 'a' "string", true "boolean", `let x;` "undefined", `{}` "object".
- `push` end, `pop` last; `length` is a property; `a[4]="x"` on 2 items → length 5. `obj.name` = `obj["name"]`.
- Parameter = name in the definition; argument = value passed. Every function returns at least `undefined`: `console.log(console.log('Hi!'))` → Hi! undefined.
- Arrows: `() => {}` · `val => "Hi " + val` (1 param, () optional) · `(a, b) => a + b`.

Scope and hoisting (L1 s115–134)

Hoisting (s132): the engine treats every `var` declaration as if it were at the top of its function scope.

	var	let (const too)
Scope	function scope	block scope
Hoisting	declaration hoisted to top: reads `undefined`	"not hoisted": error, script stops *[real: TDZ, ReferenceError: Cannot access … before initialization]*
window	global var adds `window.a`	not added (`window.b` undefined)
strict mode	re-declaration ok	re-declaration → SyntaxError

```
{ var a = 10; }  console.log(a);  // 10
{ let b = 10; }  console.log(b);  // ReferenceError: b is not defined
console.log(v);  var v = 140;     // undefined
console.log(m);  let m = 140;     // error, stops [TDZ]
const Pi = 3.14;  Pi = 90;  // TypeError: Assignment to constant
const c;                    // SyntaxError: Missing initializer
const x = {n: 1};  x.n = 2; // ok: properties still change
function f() { k = 100; }   // no var/let/const → k is global
```

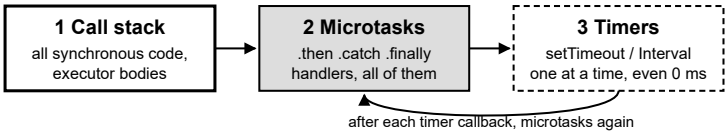
Style: **const first, then let, never var**.

Events and DOM (L1)

- Inline `<button onclick="...">`, or in a script: `btn.onclick = clickHandler;` (with `()` it runs once now and assigns `undefined`).
- `document.createElement("BUTTON")` + `document.body.appendChild(b);` `getElementsByTagName` → array; `prompt()` → input, or `null` on Cancel.

Event queue (L1 s161–162, L6 s17)

`setTimeout(f, t)` once after t ms; `setInterval(f, t)` repeats. The callback waits in the **event (callback) queue** and runs only when the **call stack is empty**, so the sync lines after it run first; 0 ms is not earlier. Promise handlers (**microtasks**) "wait only for the current call stack to unwind".




```
console.log(1);
setTimeout(function () { console.log(2); }, 1000);
console.log(3);           // 1 3 2   (same with 0 ms)

const wait = ms => new Promise(resolve => setTimeout(resolve, ms));
wait(0).then(() => console.log(2));
Promise.resolve().then(() => console.log(3))
                 .then(() => console.log(1));
console.log(4);           // 4 3 1 2
```

OOP, style, hosting (L1)

- **Inheritance** `class Car extends Vehicle { constructor(b) { super(b); } }` = **is-a**, inherits properties and methods; **composition** = **has-a**, more flexible (Car gets an Engine: swap it without changing Car). `` `Hi ${name}` ``.
- One task per function, no needless globals, no hard-coded user-facing strings (`"Your score is %1 for today"` in a separate file).
- **Hosting**: **Shared** (hundreds share; cheapest, maybe slowest; enough here) · **VPS** (fewer sites, own virtual server) · **Dedicated** (one physical machine, all yours) · **Cloud** (many machines as one). Host must give: relational DB server, several Node.js apps at once, SSL certificate.

JSON (L2 s26–29)

An object has methods, prototypes, references: it cannot travel over a network, so serialize (flatten) it. `JSON.stringify(obj)` object → string (**serialize**); `JSON.parse(str)` string → object (**deserialize**). `JSON.stringify({name: "John", age: 22})` → `'{"name":"John","age":22}'`, typeof "string".

Web Storage (L2 s30–44, L3 s7)

- **localStorage**: no expiration date. **sessionStorage**: one session, lost when the browser **tab** closes. Both in the browser, not the server.
- **Per origin** (protocol + domain + port; path does not matter; subdomain = other origin): "same origin, same browser, same machine". About 5 MB.
- **Strings only**: `getItem` → string or `null`. API: `setItem(k, v) getItem(k) removeItem(k) clear()`.
- F12 → Application → Local Storage. Password: "No. Never." (any script can read it: XSS). Token: "it can", XSS risk. Cart on every device → server side.

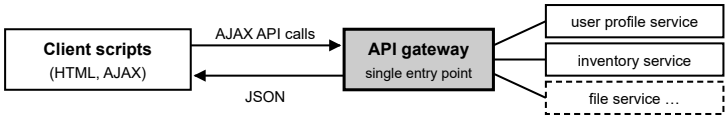
```
localStorage.setItem("list", JSON.stringify(books));  // page 1
const s = localStorage.getItem("list");   // page 2: string or null
const list = s ? JSON.parse(s) : [];      // handle the empty store
```

Architecture (L2 s4–22)

- "Architecture is not code. It is the decisions you make before you write code." Order: **Monolithic** → **Layered** → **Microservices**.
- **Monolithic**: all logic in one deployable unit, one machine, one stack. Add a feature: 1 shut down entire app, 2 add it, 3 (may need to) recompile everything. Best for prototyping, MVPs; hard to scale.

Presentation — UI, e.g. form page	Layered (n-tier)
Business — e.g. decision making, routing requests	
Application — core functions, e.g. libraries	
Data access — e.g. databases	

- **Microservices**: "a collection of loosely coupled, independently deployable services". Pros: fix one (billing) and redeploy it alone, the rest keep running (Messenger up when Facebook is down); one function → separate teams; own machine/stack. Cons: many API calls → complexity; orchestration, monitoring, careful API design.
- **Reliability** = works under normal conditions · **Robustness** = handles bad input (invalid JSON → 400 Bad Request) · **Resilience** = survives, recovers (store sells without recommendations).
- SOA ≈ MSA "in this course". Course: a simplified, API-centric MSA; API calls follow REST; client and servers on different origins; responses often JSON. Others: Event-Driven, Serverless, MVC, MVVM.



REST (L3 s44–53)

Representational State Transfer: an architectural **style / set of constraints**, not a pattern like Microservices or MVC.

1 Client–Server	separation of concerns: UI vs data/logic · evolvability (GitHub web, mobile, CLI)
2 Stateless	request carries all info, no session state: page 51, not "next page" · scalability
3 Cacheable	server marks it via Cache-Control, ETag, Expires · performance
4 Uniform Interface **(core)**	`/users/123`; nouns in URL, verbs = methods · predictability
5 Layered System	can't tell origin from proxy/gateway · flexibility, security (AWS API Gateway)

Code-on-Demand: optional 6th, on s53 only. Gateway: single entry point; routing, authentication, auditing.

Web API anatomy, GET vs POST (L3 s25–39)

Four parts: 1 **Headers** (request: API-key; response: status, content-type), 2 **Data** (body, message), 3 **Endpoint** (URL where the API server listens), 4 **HTTP Methods**: GET get, POST create, PUT update, DELETE delete. Query string `?name1=value1&name2=value2`. Web server → HTML (changes); API server → JSON/XML (does not). Map search → GET; bank login → POST.

	GET	POST
Where	query string in the URL	request body
Back/reload	harmless	data re-submitted (browser warns)
Bookmark · cache · history	can · can · params remain	cannot · never · not saved
Data length	max URL 2048 chars	no restrictions
Data type	only ASCII	binary also allowed
Security / visibility	less safe; visible in URL	"a little safer"; not in URL

Which is secure? "**none!**" (not encrypted by default; use https). No standard limit on response size for either (L6). **PUT** replaces the whole object, **PATCH** updates one property. **Idempotent** = applied many times, same result: PUT yes, POST no (repeated POST orders twice). Saving a whole edited record = PUT.

Node.js (L4)

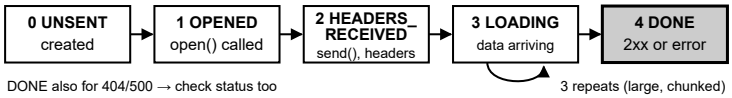
- Open-source JS runtime on Chrome V8, **outside** the browser. "Single-threaded, non-blocking, asynchronous", memory efficient *[real: event loop; thread pool reads files]*.
- **PHP, 4 steps**: send task to file system → wait while file read → return content → ready for next. **Node, 3**: send task → ready for next → return content when read. Node eliminates the waiting.
- **Restaurant**: Ex 1 one waiter serves all tables = single-threaded **non-blocking** (Node); Ex 2 waiter waits at kitchen = single-threaded **blocking** (waiter not used efficiently); Ex 3 one waiter per table = **multi-threaded** (100 requests → 100 threads).
- Modules: `exports.area = function (r) {…};` then `const mo = require('./modules/math');` (`./` same folder, `.js` optional; exported and called names must match). Built-in `http url fs`; `mysql` needs `npm install`.

```
const http = require('http');
const url = require('url');
const fs = require('fs');
const CORS = { "Access-Control-Allow-Origin": "*",
  "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
  "Access-Control-Allow-Headers": "Content-Type" };
http.createServer(function (req, res) {    // res.writeHead, not req
  if (req.method === "OPTIONS") {          // answer the preflight
    res.writeHead(204, CORS);  return res.end(); }
  const q = url.parse(req.url, true);  // true → q.query is an object
  if (q.pathname === "/time") {
    res.writeHead(200, {"Content-Type": "text/html"}); // status, not port
    res.end('<p style="color:blue">' + new Date() + '</p>');
  } else if (q.pathname === "/greet") {    // /greet?name=Ana&city=Burnaby
    res.writeHead(200, {"Content-Type": "text/plain", ...CORS});
    res.end("Hello " + q.query.name + " from " + q.query.city);
  } else fs.readFile("." + q.pathname, function (err, data) {
    if (err) { res.writeHead(404, {'Content-Type': 'text/html'});
               return res.end(q.pathname + " 404 Not Found!"); }
    res.writeHead(200, {'Content-Type': 'text/html'});
    res.write(data);  return res.end();
  });
}).listen(8888);             // http://localhost:8888/file.txt
```

`url.parse('http://localhost:8888/default.htm?name=John&age=23', true)`: `host` 'localhost:8888' · `pathname` '/default.htm' · `search` '?name=John&age=23' · `query` {name: 'John', age: '23'} (strings). Without `true`, `query` is the string → `q.query.name` undefined. **Status**: 200 OK · 3xx redirection · 4xx client error (403 Forbidden, 404 Not Found) · 5xx server error.

AJAX with XMLHttpRequest (L5 s4–9, L6 s55)

```
let xhttp = new XMLHttpRequest();            // 1 create
xhttp.onreadystatechange = function () {     // 2 handler
  if (this.readyState == 4 && this.status == 200) {
    document.getElementById("demo").innerHTML = this.responseText;
  }
};
xhttp.open("GET", "/api/echo?city=" + city, true);  // 3 (method, url, async)
xhttp.send();                        // 4 send (async true: don't wait)
```

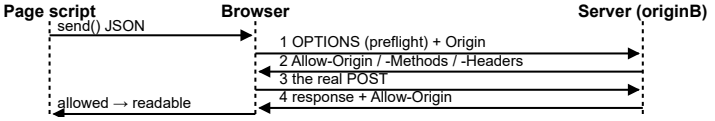


L6 s55: w3schools' "2 request received, 3 processing request" is **not accurate**.

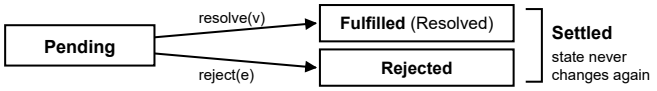
```
// POST: data in the body (GET: in the URL; a send() body is ignored)
xhttp.open("POST", endPointRoot + "patients/", true);
xhttp.setRequestHeader("Content-type", "application/x-www-form-urlencoded");
xhttp.send("name=John&age=23");
let body = "";       // server, req.method === "POST": body in chunks
req.on('data', function (chunk) { body += chunk; });
req.on('end', function () {                  // all data arrived
  const name = new URLSearchParams(body).get("name");  // string
  res.writeHead(200, {"Access-Control-Allow-Origin": "*"});
  res.end("Saved " + name); });              // reply inside 'end'
```

CORS (L5 s15–21)

- "CORS is a security feature **implemented in web browsers** and is **configured by the contacted server** in the response's header." Default: **Same-Origin Policy** (page and server scripts on one origin); browsers block cross-origin requests unless the server explicitly allows it.
- **Origin = domain + protocol + port** (80 HTTP, 443 HTTPS). `"Access-Control-Allow-Origin": "*"` (or the exact origin) in `writeHead`; also `-Allow-Methods`, `-Allow-Headers`.
- **Preflight** = method **OPTIONS**, sent first by the browser to ask if method + headers are allowed (e.g. `application/json`; not form-urlencoded or `text/plain`). Postman ignores CORS (not a browser); CORS is not encryption.



Promises (L6)



Why promises: async work finishes later, after the function has returned, and a function returns one thing; a promise carries success (resolve) or failure (reject). **Callback hell** = nesting each call in the previous callback; a `.then` chain avoids it.

1. **Order**: all sync code (executors included) → every **microtask** (handlers, in queued order, incl. ones microtasks queue) → **one timer**, its microtasks, the next timer.
2. The **executor** ("executer") runs **immediately** on `new Promise(...)` (eager).
3. **Settle once**: first resolve/reject wins; later calls ignored. Handlers run asynchronously, mutually exclusive, only once.
4. Handlers **never run synchronously**, even on an already settled promise (`Promise.resolve().then(f); log(2)` → 2 first).
5. `.then(s, f)` returns a **new promise**. **Return a value** → fulfils the next (`return new Error()` fulfils too; a returned promise is waited for). **Throw** → rejects the next.
6. A rejection skips success handlers to the next `f`/`.catch`; `.then`'s `f` watches the promise before it, not its own `s`.
7. `.catch(f)` = `.then(null, f)`; the chain is fulfilled again with `f`'s return.
8. `.finally(f)` ≈ `.then(f, f)`: runs either way, **no argument**, **passes the value/error through**, does **not** handle the rejection.
9. Non-function `.then(42)` / `.then(g())` is ignored: the value passes through.
10. No handler → `Uncaught (in promise) …` (slide: terminates *[real: browser keeps running]*). Status "resolved" *[real: "fulfilled"]*.
11. **async** function always returns a promise. **await** (inside async) waits for resolve/reject and **pauses that function only**: code after the call runs first. `try/catch` around await = `.catch`.

```
console.log("7");
let p = new Promise(function (resolve, reject) {
  resolve("4"); console.log("2"); reject("3"); console.log("5");
});
console.log("1");
p.then(m => console.log(m), e => console.log(e));
console.log("8");                  // 7 2 5 1 8 4   ("3" never)

const checkStock = qty => new Promise((resolve, reject) => {
  if (qty <= 5) resolve("reserved " + qty);
  else reject(new Error("only 5 left")); });
const findPatientP = id => new Promise(resolve => findPatient(id, resolve));
findPatientP(7)           // chain, no nesting (findVisitsP made the same way)
  .then(p => { console.log(p.name); return findVisitsP(p); })
  .then(v => console.log(v.length + " visits"))
  .catch(e => console.log(e.message))
  .finally(() => console.log("done"));
async function show(id) {          // same with async/await
  try { const p = await findPatientP(id);   // pauses show() only
        console.log(p.name);
        const v = await findVisitsP(p);  console.log(v.length + " visits");
  } catch (e) { console.log(e.message); }
}
show(7);  console.log("after");    // "after" prints before p.name
```

Node to MySQL and CRUD (L5 s31–45, L7)

```
const mysql = require("mysql");      // npm install mysql
const con = mysql.createConnection({ host: "localhost",   // 1 create
  user: "root", password: "", database: "webdev" });
con.connect(function (err) {                              // 2 connect
  if (err) throw err;
  let sql = "INSERT INTO score(name, score) " +
            "values ('elon musk', 2900)";
  con.query(sql, function (err, result) {                 // 3 query
    if (err) throw err;
    console.log("1 record inserted");
  });
});
```

CRUD	SQL	HTTP
Create (a table or DB)	INSERT (CREATE TABLE)	POST
Read	SELECT	GET
Update (records)	UPDATE	PUT (PATCH)
Delete (a record, or Drop a table)	DELETE (DROP TABLE)	DELETE

- `mysql` runs in Node, not the browser: credentials stay on the server (page code is visible to all); the page calls the API, the server runs `con.query`. **Least privilege**: one DB user per job, read user SELECT only (DROP/DELETE refused). A database is **not** a DBMS.
- **Primary key** unique per row (Patient_ID in Patient). **Foreign key** = "a primary key of another table presents in this table" (Patient_ID in Visit). 1:M patient → visits; 1:1 = UNIQUE foreign key; M:M via an associative table. **Referential integrity**: deleting a patient orphans its visits.